

بسم الله الرحمن الرحيم

Clean Code

المقدمة :

الـ Clean Code أو “الكود النظيف” هو أسلوب كتابة الأكواد البرمجية بطريقة منظمة وواضحة وسهلة القراءة والفهم. الهدف منه أن يكون الكود مفهومًا لأي مبرمج يقرأه لاحقًا، وليس فقط لمن كتبه.

-1 Clean Code :

ما هو؟

هو كتابة الكود بطريقة مرتبة وواضحة بحيث يكون:

سهل القراءة

سهل التعديل

سهل اكتشاف الأخطاء فيه

الكود النظيف يستخدم أسماء واضحة وتنظيم جيد ويبتعد عن التعقيد.

لماذا يوجد؟ وما المشكلة التي يحلها؟

عندما يكون الكود عشوائيًا أو غير منظم يصبح:

صعب الفهم

صعب التعديل

يحتوي أخطاء كثيرة

يستهلك وقتًا أطول في التطوير

نستخدم Clean Code ليساعدنا على:

تسهيل العمل الجماعي

تقليل الأخطاء

تسريع تطوير المشروع

تسهيل صيانة المشروع مستقبلاً

أمثلة على فائدته

عند عودة المبرمج للكود بعد أشهر يستطيع فهمه بسرعة.

عند عمل فريق كامل على نفس المشروع يصبح التعاون أسهل.

عند إضافة ميزة جديدة لا يحتاج المبرمج لإعادة كتابة أجزاء كثيرة.

2 - Project Structure :

ما هو؟

هو طريقة تنظيم ملفات ومجلدات المشروع حسب وظيفة كل جزء.
مثال:

مجلد للواجهات

مجلد للصور

مجلد للبيانات

مجلد للوظائف البرمجية

لماذا يوجد؟ وما المشكلة التي يحلها؟

إذا كانت الملفات مبعثرة:

يصعب العثور على الملفات

يصبح المشروع غير منظم

يضيع وقت المطورين

نستخدم Project Structure ليساعدنا على:

سهولة الوصول للملفات

فهم المشروع بسرعة

تسهيل التعديل والإضافة

أمثلة على فائدته

في تطبيق متجر إلكتروني:

مجلد للمنتجات

مجلد للمستخدمين

مجلد لعمليات الدفع

هذا يجعل كل جزء واضح ومنفصل عن الآخر.

: Namings - 3

ما هي؟

هي اختيار أسماء واضحة للمتغيرات والدوال والملفات والمجلدات.

مثال جيد:

`int totalPrice = 20`

مثال غير جيد:

`int x=20`

لماذا يوجد؟ وما المشكلة التي يحلها؟
الأسماء غير الواضحة تجعل الكود صعب الفهم.

نستخدم **Namings** ليساعدنا على:

فهم وظيفة المتغير أو الدالة مباشرة
تقليل الحاجة للتعليقات
تسهيل قراءة الكود
أمثلة على فائدتها
بدل:

`Dart`

`a = a + b`

نكتب:

`Dart`

`totalPrice = totalPrice + tax`

فيصبح الكود أوضح وأسهل للفهم.

: DRY — Don't Repeat Yourself - 4

ما هو؟

أي عدم كتابة نفس الكود أكثر من مرة، بل إنشاء دالة أو جزء مشترك واستخدامه عند الحاجة.

لماذا يوجد؟ وما المشكلة التي يحلها؟

تكرار الكود يسبب:

زيادة حجم المشروع

صعوبة التعديل

زيادة احتمالية الأخطاء

إذا تغير الكود يجب تعديله في كل مكان مكرر فيه.

لماذا نستخدمه؟

نستخدمه من أجل:

تقليل حجم الكود

تسهيل التعديل

تقليل الأخطاء

جعل المشروع أكثر تنظيماً

أمثلة على فائدته

إنشاء دالة واحدة وإعادة استخدامها بدل كتابة نفس الكود عدة مرات.

1. Clean Code

Good Example

```
double calculateTotalPrice(double price, double tax) { return price + tax;
}
```

✓ الاسم واضح

✓ الدالة قصيرة

✓ سهل فهم الكود

Bad Example

```
double x(double a, double b) { return a + b; }
```

X أسماء غير واضحة

X صعب فهم وظيفة الدالة

2. Project Structure

Good Example

```
project/ | ├── screens/ ├── models/ ├── services/ ├── images/ └──
main.dart
```

✓ الملفات مرتبة حسب الوظيفة

✓ سهل الوصول لأي ملف

Bad Example

```
project/ | ├── file1.dart ├── new.dart ├── test.dart ├── data.dart
└── abc.dart
```

X الملفات عشوائية

X صعب معرفة وظيفة كل ملف

3. Namings :

Good Example

```
String userName; int totalPrice;
```

✓ الأسماء واضحة وتشرح الغرض

Bad Example

```
String x; int a;
```

✗ أسماء غير مفهومة

✗ تجعل الكود صعب القراءة

4. DRY — Don't Repeat Yourself

Good Example

```
void showMessage() { print("Welcome"); } showMessage();  
showMessage();
```

✓ الكود غير مكرر

✓ سهل التعديل لاحقًا

Bad Example

```
print("Welcome"); print("Welcome"); print("Welcome");
```

✗ تكرار نفس الكود عدة مرات

✗ صعب التعديل إذا تغير النص لاحقًا